

CIP-B102 Foundations of Computer and Digital Forensics

Lab 7: Steganography and Hidden Data Detection

Digital Forensics Lab Report

Student Name	Fuseini Imoru Kantuogaa
Course Code	CIP-B102
Lab Title	Lab 7: Steganography and Hidden Data Detection
Date	July 15, 2026
Instructor's Name	Dr. Sarah James
Operating System Used	Kali Linux (kali@kali)

1. Pre-Lab Confirmation

I confirm that the Steganography slide walkthrough was studied in full before beginning this practical lab, including the concepts of hiding information inside files, insertion and substitution methods, least significant bit (LSB) encoding in bitmap images, steghide usage, extraction, password dictionary preparation, and stegseek.

2. Tools Used

- steghide — used to embed and extract hidden files inside the BMP carrier image.
- stegseek 0.6 — used to perform a dictionary-based password recovery attack against the stego images.
- wget — used to download the carrier image from the provided Google Drive link.
- file — used to confirm file type/format of the carrier and stego images.
- sha256sum — used to generate and compare cryptographic hashes of all evidence files.
- xxd — used to inspect the raw hexadecimal header bytes of the images.
- strings — used to search for readable text fragments inside the stego image.
- diff — used to confirm byte-for-byte equality between original and extracted files.

All tools listed above were already available on the Kali Linux lab VM and were verified as working throughout the exercise (each command executed successfully and produced the expected output shown in the procedure below).

3. Download Evidence

Source URL used:

[https://drive.google.com/file/d/1CE-4EOe9-](https://drive.google.com/file/d/1CE-4EOe9-HEF_9mg3Y4N7u15qUbIIDm0/view?usp=sharing)

[HEF_9mg3Y4N7u15qUbIIDm0/view?usp=sharing](https://drive.google.com/file/d/1CE-4EOe9-HEF_9mg3Y4N7u15qUbIIDm0/view?usp=sharing) (accessed via its direct-download form).

The working folder ~/CIP-B102-Lab7-Steganography was created first, then the carrier image was downloaded directly from the command line using wget with the direct-download URL format. The download completed successfully as an 8.8 MB file named tower_original_image_for_lab.bmp.

```

(kali㉿kali)-[~]
$ mkdir -p ~/CIP-B102-Lab7-Steganography

(kali㉿kali)-[~]
$ cd ~/CIP-B102-Lab7-Steganography

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ wget -O tower_original_image_for_lab.bmp "https://drive.google.com/uc?export=download&id=1CE-4EOe9-HEF_9mg3Y4N7u15qUbIlDm0"
--2026-07-15 12:35:36-- https://drive.google.com/uc?export=download&id=1CE-4EOe9-HEF_9mg3Y4N7u15qUbIlDm0
Resolving drive.google.com (drive.google.com) ... 142.250.129.100, 142.250.129.101, 142.250.129.102, ...
Connecting to drive.google.com (drive.google.com)|142.250.129.100|:443 ... connected.
HTTP request sent, awaiting response ... 303 See Other
Location: https://drive.usercontent.google.com/download?id=1CE-4EOe9-HEF_9mg3Y4N7u15qUbIlDm0&export=download [following]
--2026-07-15 12:35:37-- https://drive.usercontent.google.com/download?id=1CE-4EOe9-

```

Figure 1 — Working folder created and wget download of the carrier image started.

```

HEF_9mg3Y4N7u15qUbIlDm0&export=download
Resolving drive.usercontent.google.com (drive.usercontent.google.com) ... 142.251.39.193, 2a00:1450:4006:814::2001
Connecting to drive.usercontent.google.com (drive.usercontent.google.com)|142.251.39.193|:443 ... connected.
HTTP request sent, awaiting response ... 200 OK
Length: 9277494 (8.8M) [application/octet-stream]
Saving to: 'tower_original_image_for_lab.bmp'

tower_original_image 100%[=====>] 8.85M 1.15MB/s in 8.6s

2026-07-15 12:35:51 (1.03 MB/s) - 'tower_original_image_for_lab.bmp' saved [9277494/9277494]

```

Figure 2 — wget download completes successfully (200 OK, 9,277,494 bytes saved).

4. Procedure

Part A — Prepare the Working Folder and Download the Carrier Image

After the download, the directory listing, file type, and SHA-256 hash of the original carrier image were recorded before any modification, to preserve a verifiable baseline.

```

ls -lh
file tower_original_image_for_lab.bmp
sha256sum tower_original_image_for_lab.bmp > lab7_hashes.txt
cat lab7_hashes.txt

```

```

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ ls -lh
total 8.9M
-rw-rw-r-- 1 kali kali 8.9M Dec  5 2025 tower_original_image_for_lab.bmp

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ file tower_original_image_for_lab.bmp
sha256sum tower_original_image_for_lab.bmp > lab7_hashes.txt
tower_original_image_for_lab.bmp: PC bitmap, Windows 3.x format, 1365 x 2265 x 24, r
esolution 2834 x 2834 px/m, cbSize 9277494, bits offset 54

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ cat lab7_hashes.txt
6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db  tower_original_ima
ge_for_lab.bmp

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$

```

Figure 3 — Directory listing, file-type confirmation (Windows 3.x BMP, 1365x2265), and SHA-256 hash of the original image recorded in lab7_hashes.txt.

The file command confirmed the download was a genuine Windows 3.x bitmap (not an HTML error page), and the original hash 6508beb...e653c4db was captured as the baseline for later comparison.

Part B — Create the Secret Evidence Text File

A small evidence note containing my name was created with a heredoc so its exact content could be verified afterward, then its type and hash were recorded.

cat	>	secret.txt	<<	'EOF'
ICDFA	CIP-B102	Lab	7	Evidence Note
Student	Name:	Fuseini		Imoru
Message: This file was hidden and recovered during a controlled steganography lab.				
EOF				
cat				secret.txt
file secret.txt				

```

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ cat > secret.txt << 'EOF'
ICDFA CIP-B102 Lab 7 Evidence Note
Student Name: Fuseini Imoru
Message: This file was hidden and recovered during a controlled steganography lab.
EOF

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ cat secret.txt
ICDFA CIP-B102 Lab 7 Evidence Note
Student Name: Fuseini Imoru
Message: This file was hidden and recovered during a controlled steganography lab.

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ file secret.txt
secret.txt: ASCII text

```

Figure 4 — secret.txt created and its content displayed; confirmed as ASCII text.

```
sha256sum                secret.txt                >>                lab7_hashes.txt
cat lab7_hashes.txt
```

```
(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ sha256sum secret.txt >> lab7_hashes.txt

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ cat lab7_hashes.txt
6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db  tower_original_image_for_lab.bmp
b4b03641c0b06c09cc1a1274476f075e8bfdfa7e2650f9ec6e8c24d75bbc41b8  secret.txt

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$
```

Figure 5 — Hash of `secret.txt` (`b4b0364...bc41b8`) appended to `lab7_hashes.txt`.

Part C — Hide the Secret File Using Steghide

`steghide embed` was used to hide `secret.txt` inside the carrier image using LSB/substitution encoding, protected with the lab password 1234. A new file, `tower_stego_lab7.bmp`, was written so the original carrier image was never overwritten.

```
steghide  embed  -ef  secret.txt  -cf  tower_original_image_for_lab.bmp  \
-sf  tower_stego_lab7.bmp  -p  1234
ls -lh tower_original_image_for_lab.bmp tower_stego_lab7.bmp
```

```
(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ steghide embed \
  -ef secret.txt \
  -cf tower_original_image_for_lab.bmp \
  -sf tower_stego_lab7.bmp \
  -p 1234
embedding "secret.txt" in "tower_original_image_for_lab.bmp" ... done
writing stego file "tower_stego_lab7.bmp" ... done

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ ls -lh tower_original_image_for_lab.bmp tower_stego_lab7.bmp
-rw-rw-r-- 1 kali kali 8.9M Dec  5  2025 tower_original_image_for_lab.bmp
-rw-rw-r-- 1 kali kali 8.9M Jul 15 12:43 tower_stego_lab7.bmp
```

Figure 6 — `steghide embed` completes successfully; both the original and new stego image (same 8.9M visible size) are listed.

```
file                tower_stego_lab7.bmp
sha256sum            tower_stego_lab7.bmp                >>                lab7_hashes.txt
cat lab7_hashes.txt
```

```

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ file tower_stego_lab7.bmp
tower_stego_lab7.bmp: PC bitmap, Windows 3.x format, 1365 x 2265 x 24, resolution 28
34 x 2834 px/m, cbSize 9277494, bits offset 54

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ sha256sum tower_stego_lab7.bmp >> lab7_hashes.txt

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ cat lab7_hashes.txt
6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db  tower_original_ima
ge_for_lab.bmp
b4b03641c0b06c09cc1a1274476f075e8bfd7a7e2650f9ec6e8c24d75bbc41b8  secret.txt
c7669807d6bce5ceb52c3045e6149b40d89002ba903228c66092cbb8a2e55d23  tower_stego_lab7.b
mp

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$

```

Figure 7 — Stego image confirmed as a valid BMP of identical dimensions; its hash (c766980...2e55d23) is different from the original image's hash.

Explanation: although the reported file size and image dimensions are unchanged, embedding secret.txt altered the least-significant bits of many pixel bytes. Because SHA-256 is sensitive to any single-bit change, the stego image hash differs completely from the original image hash even though the two files look visually identical.

Part D — Extract the Hidden File Using the Correct Password

The hidden file was extracted from the stego image with the correct password, and the recovered content was displayed and compared against the original secret file.

mkdir		-p		extracted
steghide	extract	-sf	tower_stego_lab7.bmp	\
	-xf	extracted/extracted_secret.txt	-p	1234
cat extracted/extracted_secret.txt				

```

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ mkdir -p extracted

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ steghide extract \
-sf tower_stego_lab7.bmp \
-xf extracted/extracted_secret.txt \
-p 1234
wrote extracted data to "extracted/extracted_secret.txt".

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ cat extracted/extracted_secret.txt
ICDFA CIP-B102 Lab 7 Evidence Note
Student Name: Fuseini Imoru
Message: This file was hidden and recovered during a controlled steganography lab.

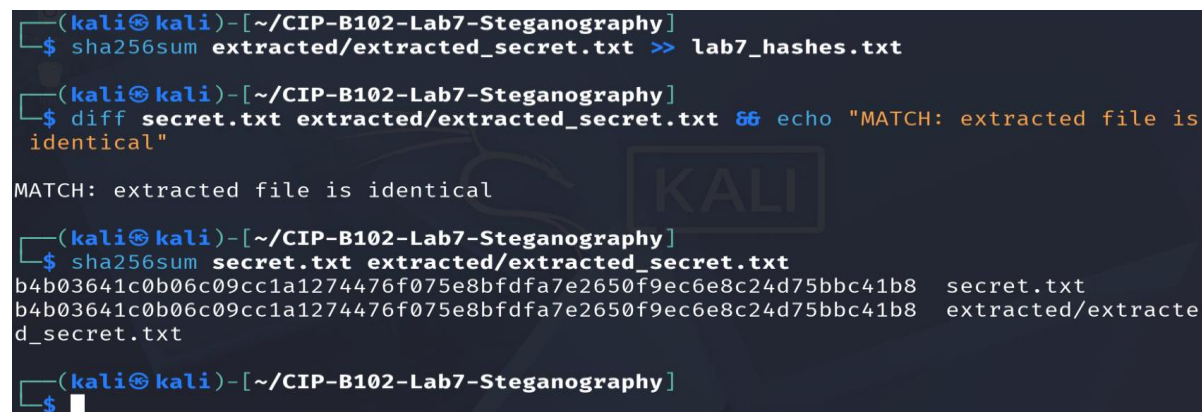
```

Figure 8 — steghide extract recovers the hidden data; the recovered text matches the original evidence note exactly.

```

sha256sum          extracted/extracted_secret.txt          >>          lab7_hashes.txt
diff secret.txt extracted/extracted_secret.txt && echo "MATCH: extracted file is identical"
sha256sum secret.txt extracted/extracted_secret.txt

```



```

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ sha256sum extracted/extracted_secret.txt >> lab7_hashes.txt

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ diff secret.txt extracted/extracted_secret.txt && echo "MATCH: extracted file is identical"
MATCH: extracted file is identical

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ sha256sum secret.txt extracted/extracted_secret.txt
b4b03641c0b06c09cc1a1274476f075e8bfdfa7e2650f9ec6e8c24d75bbc41b8  secret.txt
b4b03641c0b06c09cc1a1274476f075e8bfdfa7e2650f9ec6e8c24d75bbc41b8  extracted/extracted_secret.txt

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$

```

Figure 9 — *diff* reports *MATCH*, and *sha256sum* confirms *secret.txt* and the extracted file share the identical hash *b4b0364...bc41b8*.

This proves the recovered file is a perfect, forensically sound copy of the original hidden evidence no data was lost or altered during embedding or extraction.

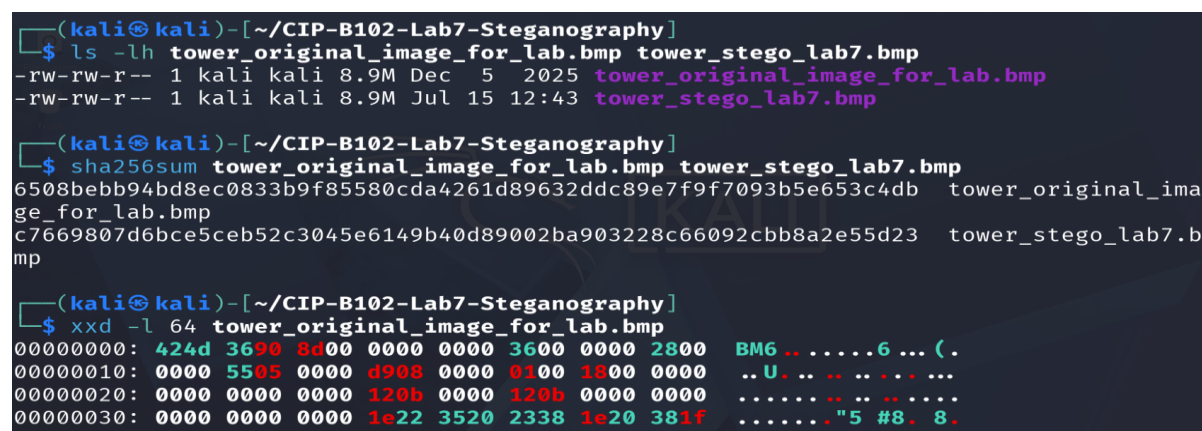
Part E — Basic Visual and Binary Comparison

The original and stego images were compared at the file-system, hash, and raw-byte level to demonstrate that a visually identical image can carry different underlying data.

```

ls          -lh          tower_original_image_for_lab.bmp          tower_stego_lab7.bmp
sha256sum          tower_original_image_for_lab.bmp          tower_stego_lab7.bmp
xxd -l 64 tower_original_image_for_lab.bmp

```



```

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ ls -lh tower_original_image_for_lab.bmp tower_stego_lab7.bmp
-rw-rw-r-- 1 kali kali 8.9M Dec  5  2025 tower_original_image_for_lab.bmp
-rw-rw-r-- 1 kali kali 8.9M Jul 15 12:43 tower_stego_lab7.bmp

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ sha256sum tower_original_image_for_lab.bmp tower_stego_lab7.bmp
6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db tower_original_image_for_lab.bmp
c7669807d6bce5ceb52c3045e6149b40d89002ba903228c66092cbb8a2e55d23 tower_stego_lab7.bmp

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ xxd -l 64 tower_original_image_for_lab.bmp
00000000: 424d 3690 8d00 0000 0000 3600 0000 2800  BM6 .. ....6 ... ( .
00000010: 0000 5505 0000 d908 0000 0100 1800 0000  .. U. . . . .
00000020: 0000 0000 0000 120b 0000 120b 0000 0000  ..... .. .
00000030: 0000 0000 0000 1e22 3520 2338 1e20 381f  .... "5 #8. 8.

```

Figure 10 — *File sizes are identical, but the SHA-256 hashes differ; first 64 bytes of the original image's BMP header shown with *xxd*.*

```

xxd                                -l          64                                tower_stego_lab7.bmp
strings tower_stego_lab7.bmp | head -n 20

```

```

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ xxd -l 64 tower_stego_lab7.bmp
00000000: 424d 3690 8d00 0000 0000 3600 0000 2800  BM6 .. ...6 ... (.
00000010: 0000 5505 0000 d908 0000 0100 1800 0000  ..U. . . . .
00000020: 0000 0000 0000 120b 0000 120b 0000 0000  ..... .. .
00000030: 0000 0000 0000 1e22 3520 2338 1e20 381f  ..... "5 #8. 8.

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ strings tower_stego_lab7.bmp | head -n 20
"5 #8
:$(?)E')H**M*,N).O(.Q'0R(1S(2T/9[5Ac=NoDYyF_
B_~;Wv7Nn2Cd2>`1:\17Z79\79\;;_;;_;;`79\67]56\56\67]67]56\56\56\79\57Z46Y57Z46Y68[68[
35X56X23U23U45W23U12T34V34V56X45W34V-.P./Q,-O/0R32R10P/.N.-M/.N/.N..L--K//M++I..L
--K,-I,-I**G**G),H8)E8)E'*F8*C'+D)*D')A(*B8(@')A')A8)>8)>*>$'<+,A+,A**@(>)'(=((>)
( )=( )=' );%'9#%7#%7#%7"$6$888)8$'6%%5%%5##3##3$4!!1! /
#!#+-/?7BGMPNTNPQJLLFHHHJJIKKJLLNNIKKBDD?AA;:<?>@EDF?>@=<?>@;=<=<:≤578:≤6899;<9;
<@ABDEJNOKOPMOPLNOIJNJKOORVRUYRWZY^adknlsvrx}msxZ^cJNS/47$),

```

Figure 11 — First 64 bytes of the stego image's BMP header (identical structure to the original), followed by the start of the strings output on the pixel data.

```

'0> )78/<5>K3=G*4>5?IBLVE0YIS]0YcPZdP\hQ]iP\hMYeFR^<HT<HTEQ]IUaLXd0[gNZfJVbGS_FR^GS_
GS_7CO;/G4@L6BN-9E'3?(4@*6B.:F3?K:FRAMYDP\EQ]DP\FPZEOYEN[EN[FO\HQ^HQ^DMZBLVDNXGQ[GQ[
FPZHR\IR_IR_HQ_GP^FO]EN\DM[DM[EN\GP^KtIR_FO\FO\GP]FO\EN[GP]FO\DMZCLYEN[IR_KTaIR_HQ^
JS`JS`KtAKTaLUBPYfR[h0XeMV`PYcU^hYbLZcm\eo]fp\eo[dn\eo_hrbkudmwhq{ir|gpzjs}kt~lt

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ steghide info tower_stego_lab7.bmp
"tower_stego_lab7.bmp":
  format: Windows 3.x bitmap
  capacity: 377.4 KB
Try to get information about embedded data ? (y/n) y
Enter passphrase:
  embedded file "secret.txt":
    size: 146.0 Byte
    encrypted: rijndael-128, cbc
    compressed: yes

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$

```

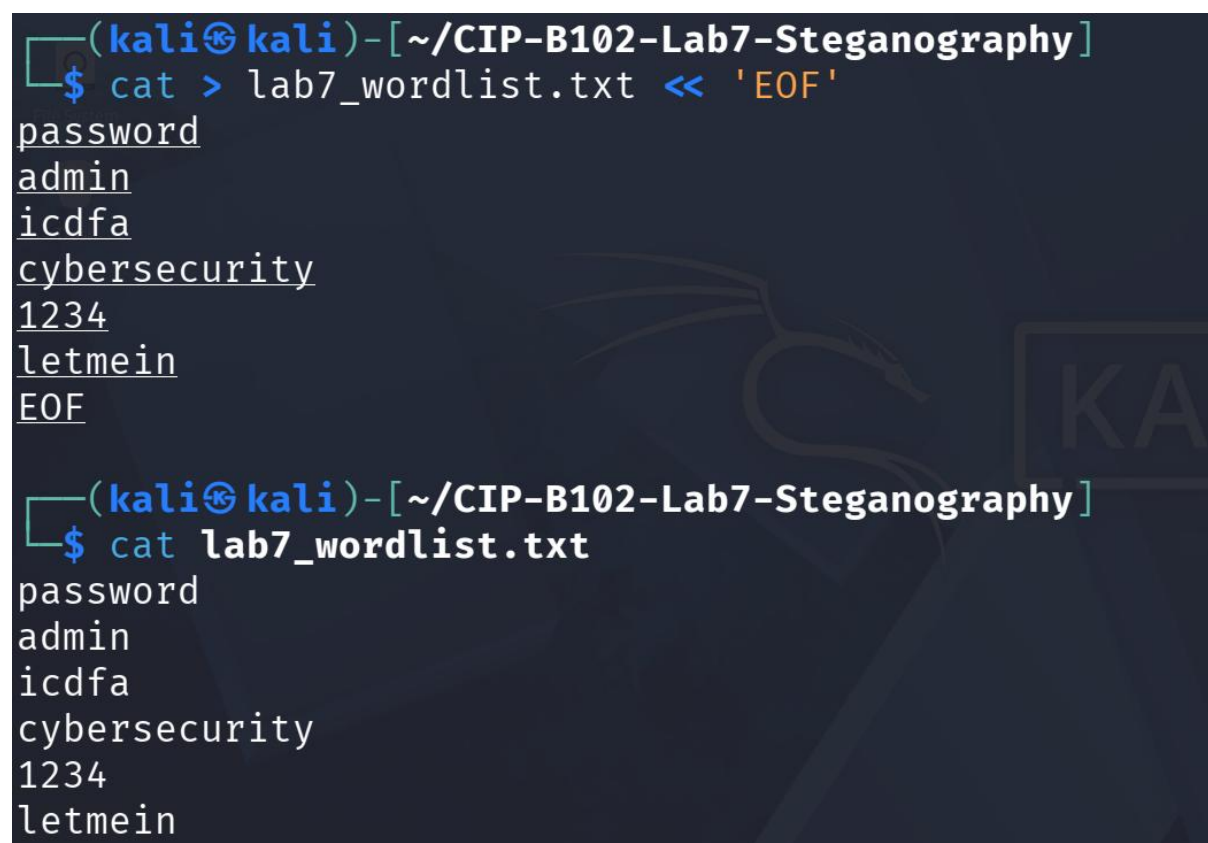
Figure 12 — Continued strings output showing unreadable, high-entropy pixel data, followed by steghide info confirming a 146-byte AES(rijndael-128)-encrypted, compressed payload named secret.txt embedded in the file.

Interpretation: the BMP headers (first 64 bytes) of the two files are byte-for-byte identical, and the strings output shows no readable evidence of the hidden file — the pixel data simply reads as noise either way. This confirms that neither the header nor a visual/textual inspection is sufficient to detect steganography; only a hash comparison or a tool such as steghide/stegseek can reveal that hidden data is present.

Part F — Password Dictionary Test with Stegseek

A small custom wordlist containing the known lab password (1234) alongside several common weak passwords was created, then stegseek was run against the stego image to test whether the password could be recovered automatically.

```
cat > lab7_wordlist.txt << 'EOF'
password
admin
icdfa
cybersecurity
1234
letmein
EOF
cat lab7_wordlist.txt
```



```
(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ cat > lab7_wordlist.txt << 'EOF'
password
admin
icdfa
cybersecurity
1234
letmein
EOF

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ cat lab7_wordlist.txt
password
admin
icdfa
cybersecurity
1234
letmein
```

Figure 13 — Custom lab7_wordlist.txt created and displayed, containing six candidate passwords including 1234.

```
stegseek                                tower_stego_lab7.bmp                                lab7_wordlist.txt
ls -lh
```

```
(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ stegseek tower_stego_lab7.bmp lab7_wordlist.txt
StegSeek 0.6 - https://github.com/RickdeJager/StegSeek

[i] Found passphrase: "1234"
[i] Original filename: "secret.txt".
[i] Extracting to "tower_stego_lab7.bmp.out".

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ ls -lh
total 18M
drwxrwxr-x 2 kali kali 4.0K Jul 15 12:46 extracted
-rw-rw-r-- 1 kali kali 360 Jul 15 12:46 lab7_hashes.txt
-rw-rw-r-- 1 kali kali 48 Jul 15 12:55 lab7_wordlist.txt
-rw-rw-r-- 1 kali kali 146 Jul 15 12:39 secret.txt
-rw-rw-r-- 1 kali kali 8.9M Dec 5 2025 tower_original_image_for_lab.bmp
-rw-rw-r-- 1 kali kali 8.9M Jul 15 12:43 tower_stego_lab7.bmp
-rw-rw-r-- 1 kali kali 146 Jul 15 12:56 tower_stego_lab7.bmp.out
```

Figure 14 — stegseek 0.6 finds the passphrase "1234" almost instantly, identifies the original filename secret.txt, and extracts it to tower_stego_lab7.bmp.out.

Explanation: because the embedding password (1234) was included in the six-word test dictionary, stegseek recovered it in a fraction of a second and automatically extracted the hidden file without steghide's password even being supplied manually. This demonstrates why weak or common passwords are dangerous in steganography any password that appears in a small, easily-guessed wordlist (let alone a large real-world list such as rockyou.txt) can be defeated almost instantly by automated dictionary attacks. A strong, unique passphrase is essential for the hidden data to resist this kind of attack.

Part G — Independent Student Task

A separate steganography evidence file was created, embedded, extracted, and password-tested independently, following the same workflow as Parts B–F but with my own content and stego image.

```
cat > fuseini_imoru_hidden_note.txt << 'EOF'
Steganography is the practice of hiding information inside another file, such as an
image, audio file, or video file.
In digital forensics, steganography matters because investigators may encounter files
```

that appear normal but secretly contain hidden evidence, confidential information, or malicious content. Understanding how to detect and recover hidden data is an important forensic skill.

EOF

```
(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ cat > fuseini_imoru_hidden_note.txt << 'EOF'
Steganography is the practice of hiding information inside another file, such as an
image, audio file, or video file.

In digital forensics, steganography matters because investigators may encounter file
s that appear normal but secretly contain hidden evidence, confidential information,
or malicious content. Understanding how to detect and recover hidden data is an imp
ortant forensic skill.
EOF

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ cat fuseini_imoru_hidden_note.txt
Steganography is the practice of hiding information inside another file, such as an
image, audio file, or video file.

In digital forensics, steganography matters because investigators may encounter file
s that appear normal but secretly contain hidden evidence, confidential information,
or malicious content. Understanding how to detect and recover hidden data is an imp
```

Figure 15 — Independent evidence note `fuseini_imoru_hidden_note.txt` created, explaining what steganography is and why it matters in forensics.

```
steghide          embed          -ef          fuseini_imoru_hidden_note.txt          \
-cf  tower_original_image_for_lab.bmp  -sf  tower_stego_partG.bmp  -p  1234
sha256sum fuseini_imoru_hidden_note.txt
```

```
(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ steghide embed \
  -ef fuseini_imoru_hidden_note.txt \
  -cf tower_original_image_for_lab.bmp \
  -sf tower_stego_partG.bmp \
  -p 1234
embedding "fuseini_imoru_hidden_note.txt" in "tower_original_image_for_lab.bmp" ... 0
embedding "fuseini_imoru_hidden_note.txt" in "tower_original_image_for_lab.bmp" ... d
one
writing stego file "tower_stego_partG.bmp" ... done

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ sha256sum fuseini_imoru_hidden_note.txt
d51d6b45d8504a48de9144bd885adaae98315ace6ffd862195593b4b14b3cfa5  fuseini_imoru_hidd
en_note.txt
```

Figure 16 — Independent note embedded into a new stego image `tower_stego_partG.bmp`; hash of the original note recorded (`d51d6b4...b3cfa5`).

```

sha256sum                                     tower_stego_partG.bmp
mkdir                                          -p                                     partG_extract
steghide          extract          -sf          tower_stego_partG.bmp          \
-xf partG_extract/recovered_note.txt -p 1234

```

```

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ sha256sum tower_stego_partG.bmp
3c9def718a7d0e58dbe27cf7b192224354d6261cd756f601316454bf4491b57d tower_stego_partG.
bmp

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ mkdir -p partG_extract

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ steghide extract \
-sf tower_stego_partG.bmp \
-xf partG_extract/recovered_note.txt \
-p 1234
wrote extracted data to "partG_extract/recovered_note.txt".

```

Figure 17 — Hash of the stego image recorded, then the hidden note successfully extracted to `partG_extract/recovered_note.txt`.

```

sha256sum          fuseini_imoru_hidden_note.txt          partG_extract/recovered_note.txt
diff fuseini_imoru_hidden_note.txt partG_extract/recovered_note.txt && echo "MATCH:
files are identical"

```

```

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ sha256sum fuseini_imoru_hidden_note.txt
d51d6b45d8504a48de9144bd885adaae98315ace6ffd862195593b4b14b3cfa5 fuseini_imoru_hidd
en_note.txt

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ sha256sum partG_extract/recovered_note.txt
d51d6b45d8504a48de9144bd885adaae98315ace6ffd862195593b4b14b3cfa5 partG_extract/reco
vered_note.txt

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ diff fuseini_imoru_hidden_note.txt partG_extract/recovered_note.txt \
&& echo "MATCH: files are identical"
MATCH: files are identical

```

Figure 18 — The original and recovered note share the identical hash `d51d6b4...b3cfa5`, and `diff` confirms `MATCH`.

```

cat          >          partG_wordlist.txt          <<          'EOF'
password
admin
forensics

```

```
cybersecurity
1234
letmein
EOF
stegseek tower_stego_partG.bmp partG_wordlist.txt
```

```
(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ cat > partG_wordlist.txt << 'EOF'
password
admin
forensics
cybersecurity
1234
letmein
EOF

(kali㉿kali)-[~/CIP-B102-Lab7-Steganography]
$ stegseek tower_stego_partG.bmp partG_wordlist.txt
StegSeek 0.6 - https://github.com/RickdeJager/StegSeek

[i] Found passphrase: "1234"
[i] Original filename: "fuseini_imoru_hidden_note.txt".
[i] Extracting to "tower_stego_partG.bmp.out".
```

Figure 19 — A second mini dictionary was tested against tower_stego_partG.bmp; stegseek again recovers the passphrase "1234" and correctly identifies the original filename.

Note on password handling: the embedding password used for the independent task is recorded here and in my private notes as 1234 (chosen so the recovery step could be verified); per the lab instructions this is not presented as a "secret" on the report cover page.

5. Hash Table Summary

The following SHA-256 hashes were recorded in lab7_hashes.txt and verified throughout the exercise:

File	Stage	SHA-256 Hash
tower_original_image_for_lab.bmp	Original carrier	6508bebb94bd8ec0833b9f85580cda4
	image	261d89632ddc89e7f9f7093b5e653c4db
secret.txt	Original secret	b4b03641c0b06c09cc1a1274476f075
	file (Part B)	e8bfdfa7e2650f9ec6e8c24d75bbc41b8

tower_stego_lab7.bmp	Stego image after embedding (Part C)	c7669807d6bce5ceb52c3045e6149b40d89002ba903228c66092cbb8a2e55d23
extracted/extracted_secret.txt	Extracted file (Part D)	b4b03641c0b06c09cc1a1274476f075e8bfdfa7e2650f9ec6e8c24d75bbc41b8
fuseini_imoru_hidden_note.txt	Independent task original note (Part G)	d51d6b45d8504a48de9144bd885adaae98315ace6ffd862195593b4b14b3cfa5
tower_stego_partG.bmp	Independent task stego image (Part G)	3c9def718a7d0e58dbe27cf7b192224354d6261cd756f601316454bf4491b57d
partG_extract/recovered_note.txt	Independent task recovered file (Part G)	d51d6b45d8504a48de9144bd885adaae98315ace6ffd862195593b4b14b3cfa5

6. Findings

- The hidden file was recovered successfully in every case: the guided Part D extraction and the independent Part G extraction both produced files that are byte-for-byte identical to their originals (confirmed by diff and matching SHA-256 hashes).
- Embedding data with steghide changed the SHA-256 hash of the carrier image (Part C/E) without changing its visible dimensions, apparent file size on disk, or the readable structure of its BMP header confirming that hash comparison, not visual inspection, is the reliable way to detect that a file has been modified.
- Both the guided stego image (password 1234) and the independently created stego image (also password 1234) were defeated instantly by stegseek once that password was present in a small custom wordlist.

7. Password Test Results

stegseek 0.6 was run against both stego images produced in this lab. In both cases it recovered the passphrase "1234" from a six-entry custom wordlist in well under a second, correctly identified the original embedded filename, and automatically extracted the hidden payload. No

installation or execution failures were encountered, so no fallback/troubleshooting steps were required.

8. Reflection

This lab demonstrated that steganography hides the existence of data rather than encrypting its content unlike encryption, which scrambles data so it is unreadable but still visibly present, steganography conceals data inside another file (here, the least-significant bits of a bitmap image) so that no obvious evidence of hidden information exists at all. The exercise showed that a stego image can be dimensionally and visually identical to its source while carrying a completely different hash, which is why forensic examiners rely on cryptographic hashing, header/structure analysis, and dedicated tools such as steghide and stegseek rather than the naked eye when screening evidence for hidden data. The stegseek results in Parts F and G were a clear illustration of why password strength matters: a four-digit numeric password was recovered from a six-word dictionary almost instantly, and would be recovered just as easily from a large real-world wordlist such as rockyou.txt. In an actual investigation, being able to test suspect files against common wordlists and knowing that weak passphrases offer negligible real protection is directly relevant to uncovering concealed evidence, malicious payloads, or exfiltrated data hidden inside seemingly ordinary image files.

9. Reflection Questions

1. What is steganography and how is it different from encryption?

Steganography hides the existence of data by concealing it inside another file so it is not apparent that any hidden data exists. Encryption instead scrambles data so it cannot be read without a key, but the presence of encrypted data is usually still visible.

2. Why are BMP images commonly used when explaining LSB steganography?

BMP files store pixel data uncompressed, so each pixel's least-significant bits can be modified predictably without a compression algorithm re-encoding or discarding the changes, making the LSB substitution technique easy to demonstrate and reverse.

3. Why does the stego image hash change after embedding a file?

Embedding alters the least-significant bits of numerous pixel bytes throughout the image. SHA-256 produces a completely different digest for any change, even a single bit, so the stego image's hash no longer matches the original.

4. Can a normal visual inspection prove whether an image contains hidden data? Explain.

No. As shown in Part E, the original and stego images were visually and dimensionally identical, and their BMP headers matched byte-for-byte; only a hash comparison or a steganalysis tool revealed the difference.

5. Why is password strength important when using steghide?

As demonstrated in Parts F and G, a weak four-digit password was recovered by stegseek in under a second using a tiny custom wordlist. A weak password gives essentially no real protection against automated dictionary attacks.

6. How can steganography be relevant in digital forensic investigations?

Suspects may use steganography to conceal evidence, stolen data, or communications inside ordinary-looking image, audio, or video files. Investigators must know how to detect (via hashing/steganalysis) and, where a weak password is used, recover such hidden data during an examination.

10. Appendix — Full Command Log

```
mkdir                                -p                                ~/CIP-B102-Lab7-Steganography
cd                                  ~/CIP-B102-Lab7-Steganography
wget                                -O                                tower_original_image_for_lab.bmp
"https://drive.google.com/uc?export=download&id=1CE-4EOe9-
HEF_9mg3Y4N7u15qUbIIDm0"
ls                                  -lh
file                                tower_original_image_for_lab.bmp
sha256sum                          tower_original_image_for_lab.bmp    >                                lab7_hashes.txt
cat                                lab7_hashes.txt

cat                                >                                secret.txt                        <<                                'EOF'
ICDFA                             CIP-B102                         Lab                               7                               Evidence                         Note
Student                           Name:                           Fuseini                          Imoru
Message: This file was hidden and recovered during a controlled steganography lab.
EOF
cat                                secret.txt
file                                secret.txt
```

```

sha256sum                secret.txt                >>                lab7_hashes.txt

steghide embed -ef secret.txt -cf tower_original_image_for_lab.bmp -sf tower_stego_lab7.bmp
-p                                                                1234
ls            -lh            tower_original_image_for_lab.bmp            tower_stego_lab7.bmp
file                                                tower_stego_lab7.bmp
sha256sum            tower_stego_lab7.bmp            >>                lab7_hashes.txt

mkdir                                -p                                extracted
steghide  extract  -sf  tower_stego_lab7.bmp  -xf  extracted/extracted_secret.txt  -p  1234
cat                                                extracted/extracted_secret.txt
sha256sum            extracted/extracted_secret.txt            >>                lab7_hashes.txt
diff secret.txt extracted/extracted_secret.txt && echo "MATCH: extracted file is identical"
sha256sum            secret.txt            extracted/extracted_secret.txt

sha256sum            tower_original_image_for_lab.bmp            tower_stego_lab7.bmp
xxd            -l            64            tower_original_image_for_lab.bmp
xxd            -l            64            tower_stego_lab7.bmp
strings            tower_stego_lab7.bmp            |            head            -n            20
steghide                                info                                tower_stego_lab7.bmp

cat            >            lab7_wordlist.txt            <<            'EOF'
password  /  admin  /  icdfa  /  cybersecurity  /  1234  /  letmein
EOF
stegseek            tower_stego_lab7.bmp            lab7_wordlist.txt

cat            >            fuseini_imoru_hidden_note.txt            <<            'EOF'
(independent            evidence            note            text)
EOF
steghide embed -ef fuseini_imoru_hidden_note.txt -cf tower_original_image_for_lab.bmp -sf
tower_stego_partG.bmp            -p            1234
sha256sum            fuseini_imoru_hidden_note.txt
sha256sum            tower_stego_partG.bmp
mkdir                                -p                                partG_extract

```

```
steghide extract -sf tower_stego_partG.bmp -xf partG_extract/recovered_note.txt -p 1234
sha256sum      fuseini_imoru_hidden_note.txt      partG_extract/recovered_note.txt
diff fuseini_imoru_hidden_note.txt partG_extract/recovered_note.txt && echo "MATCH:
files          are                                identical"
cat            >      partG_wordlist.txt          <<      'EOF'
password / admin / forensics / cybersecurity / 1234 / letmein
EOF
stegseek tower_stego_partG.bmp partG_wordlist.txt
```

No error messages were encountered during this lab; every command listed above completed successfully as shown in the corresponding figures.